
Software Design and Architecture

*Habtam D.
AASTU, 2022*

Chapter Four

Design Patterns

Design Patterns

- **Pattern** is a **solution** to a **problem** in a **context**.
 - » **Context**: the situation in which the pattern apply
 - » **Problem**: goal you are trying to achieve in this context (any constraints that occur in that context)
 - » **Solution**: what you are after – general design that anyone can apply which resolves the goal and set of constraints.

Cont'd

- ***Design Patterns*** - descriptions of communicating objects and classes that are customized **to solve a general design problem in a particular context.**
 - » A problem that someone has already solved.
 - » A model or design to use as a guide.
 - » **More formally:** "A proven solution to a common problem in a specified context."
- **Real World Examples:**
 - » Blueprint for a house
 - » Manufacturing

Cont'd

- In software engineering, a design pattern is a **general repeatable solution to a commonly occurring problem** in software design.
- A ***design pattern is*** a standardized solution to a problem commonly encountered during object-oriented software development.
- A design pattern ***is not*** a finished design that can be transformed directly into code.
 - » It is a **description or template for how to solve a problem** that can be used in many different situations.
 - » Design patterns help designers to choose design alternatives that make a system reusable and avoid alternatives that compromise reusability.
 - » Design patterns can even improve the documentation and maintenance of existing systems by furnishing an explicit specification of class and object interactions and their underlying intent.

History of Design Patterns

- Christopher Alexander (Civil Engineer) in 1977 wrote
 - » “Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, *without ever doing it the same way twice.*”
- In 1995, the principles that Alexander established were applied to software design and architecture. The result was the book:
“Design Patterns: Elements of Reusable Object-Oriented Software” by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides.
- Also commonly known as **“The Gang of Four”**.

What is Gang of Four (GoF)?

- In 1995, four authors **Erich Gamma**, **Richard Helm**, **Ralph Johnson** and **John Vlissides** published a book titled **Design Patterns - Elements of Reusable Object-Oriented Software** which initiated the concept of Design Pattern in Software development.
- These authors are collectively known as **Gang of Four (GOF)**.
- According to these authors **design patterns are primarily based on the following principles of object orientated design.**
 - Program to an interface not an implementation
 - Favor object composition over inheritance

The Gang of Four

- Defines a Catalog of different design patterns.
- Three different types
 - » **Creational** – “creating objects in a manner suitable for the situation”.
 - These design patterns **provide a way to create objects while hiding the creation logic**, rather than instantiating objects directly using new operator. This gives program more flexibility in deciding which objects need to be created for a given use case.
 - » **Structural** – “ease the design by identifying a simple way to realize relationships between entities”.
 - These design patterns **concern class and object composition**. Concept of inheritance is used to compose interfaces and define ways to compose objects to obtain new functionalities.
 - » **Behavioral** – “common communication patterns between objects”.
 - These design patterns are specifically concerned with communication between objects.

The Gang of Four: Pattern Catalog

Creational

- Abstract Factory
- Builder
- Factory Method**
- Prototype
- Singleton

Structural

- Adapter**
- Bridge
- Composite
- Decorator
- Façade
- Flyweight
- Proxy

Behavioral

- Chain of Responsibility
- Command
- Interpreter
- Iterator
- Mediator
- Memento
- Observer
- State
- Strategy
- Template Method
- Visitor

Characteristics of Design Patterns

- **Viewpoints** – ways to describe patterns.
 - » Static: class model (building blocks).
 - » Dynamic: sequence or state diagram (operation).
- **Levels** – decomposition of patterns.
 - » Abstract level describes the core of the pattern.
 - » Concrete (= non abstract) level describes the particulars of this case.
- **Roles** – the “players” in pattern usage.
 - » Application of the design pattern itself.
 - » Clients of the design pattern application.
 - » Setup code initializes and controls.

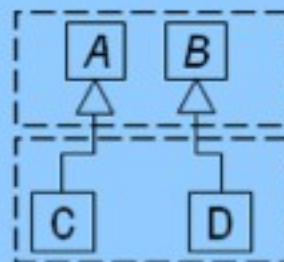
Cont'd

(class or classes)

1. Client role

3. Role: Application of the design pattern

A. Static viewpoint

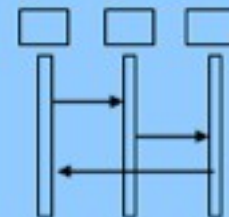


(i) Abstract level

(ii) Concrete level

(class model)

B. Dynamic viewpoint



(sequence or
state diagram)

2. Setup role

(class or classes)

-----> : Reference direction

Design Pattern Forms

- **Delegation**

- » An implementation mechanism in which an object forwards (delegates) a request to another object.
- » The delegate carries out the request on behalf of the original object.

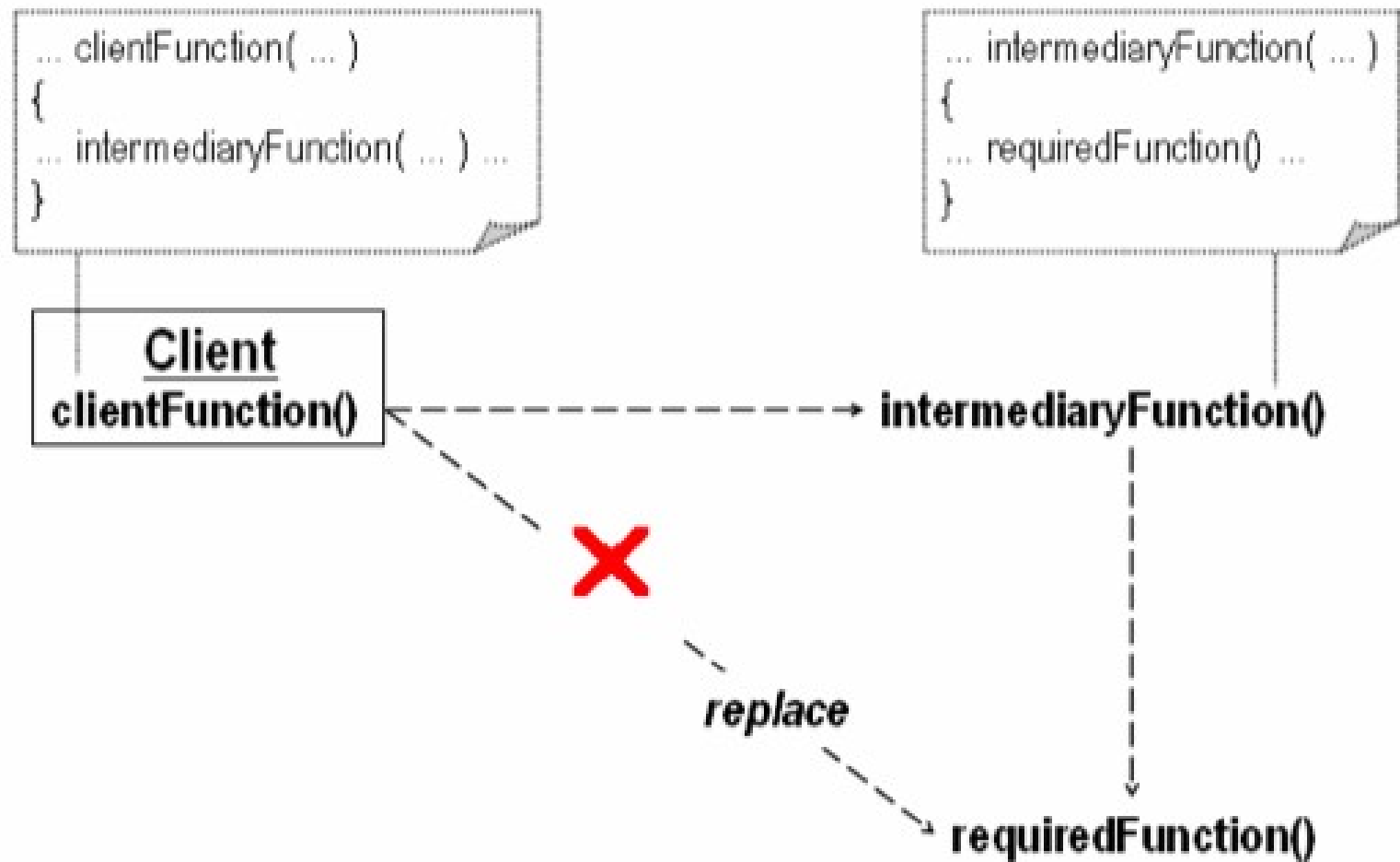
- **Recursion**

- » Part of the pattern essentially uses itself.

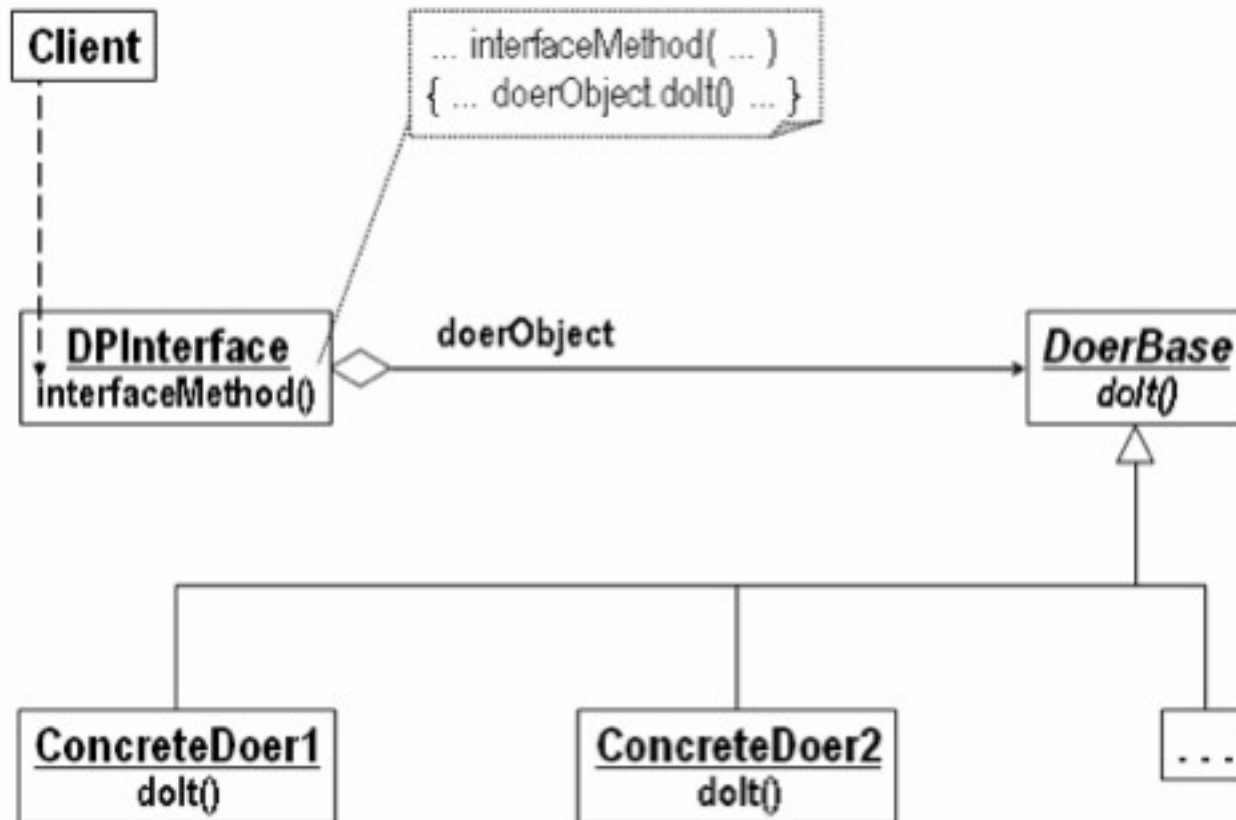
- ***Example***

- Constructing graphical user interfaces that allow for windows within window.

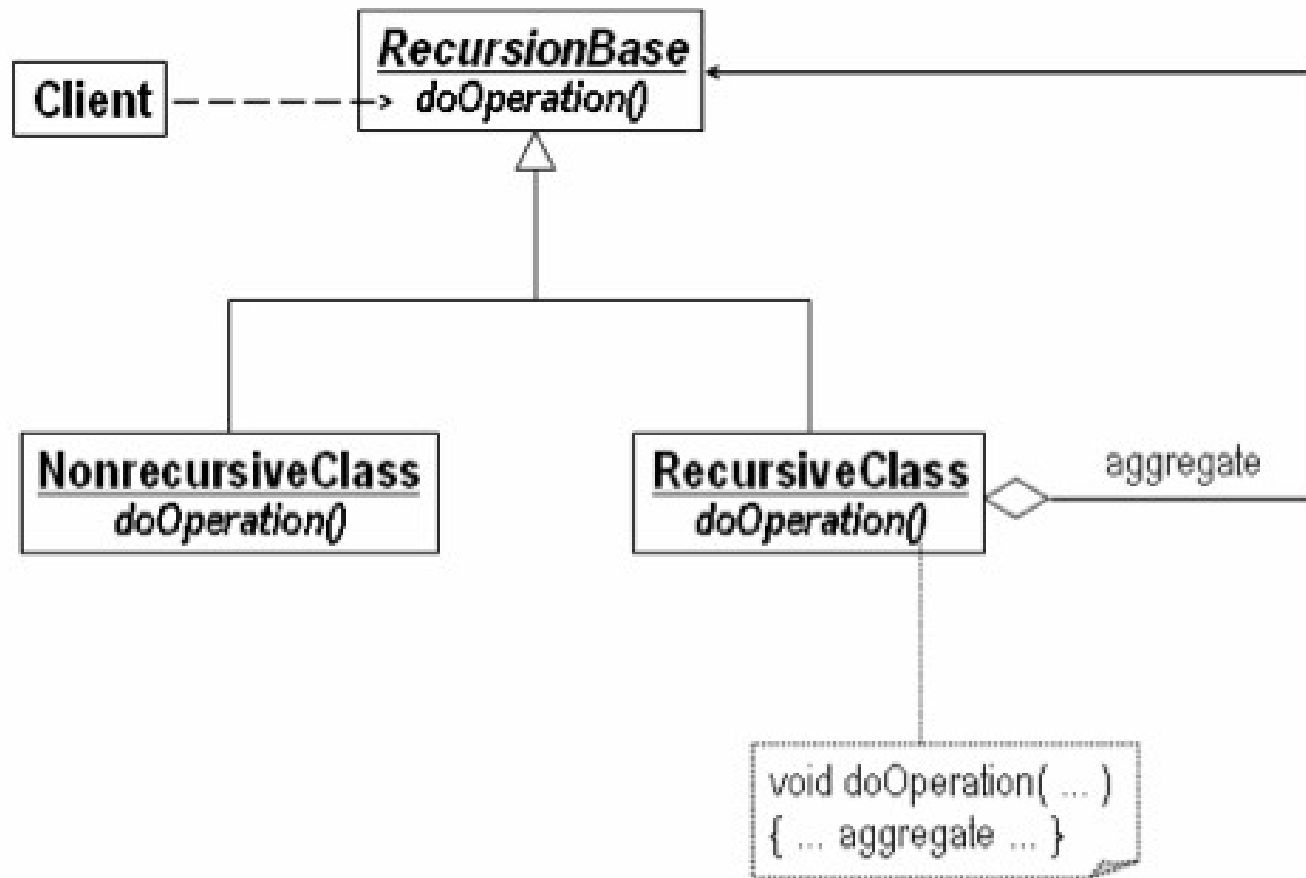
Delegation



Cont'd



Recursion



Design Pattern Goals

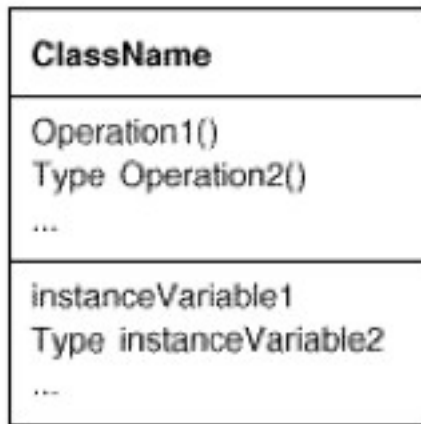
- To support reuse of successful designs.
- To facilitate software evolution.
 - » Add new features easily, without breaking existing ones
- In short, we want to design for change.
- **Principles to achieve the above goals:**
 - » Reduce implementation dependencies between elements of a software system
 - Program to an interface, not an implementation
 - Favor composition over inheritance
 - Use delegation

Elements of Design Patterns

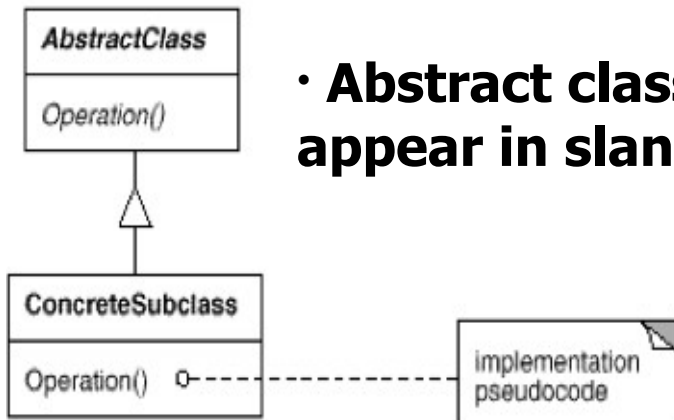
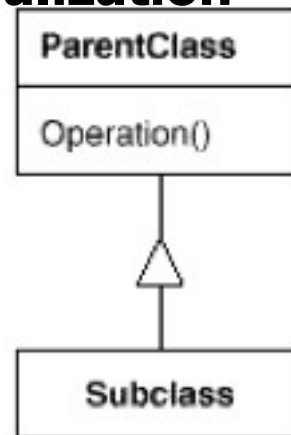
- In general, a pattern has four basic elements:
 - » **Pattern name**- is a handle we can use to describe a design problem, its solutions, and consequences in a word or two.
 - » **Problem** - When to apply the pattern. It explains the problem and its context.
 - » **Solution** - Elements that make up the pattern, their relationships, responsibilities, and collaborations.
 - » **Consequences** - Results and trade-offs of applying the pattern.

Basic UML/OMT Notations in the GOF Patterns

Class Diagram



Generalization/ Specialization



- **Abstract classes appear in slanted type**

Object Instantiation

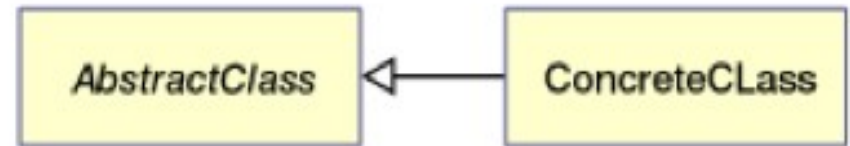


Class Association UML/OMT Notations

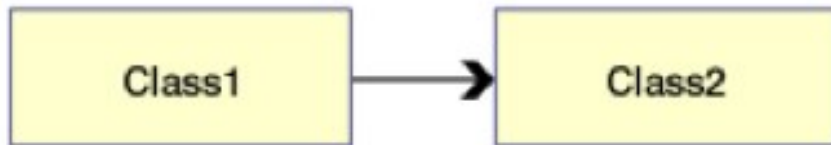
Association



Specialization



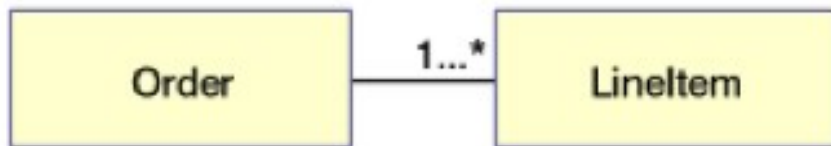
Navigability



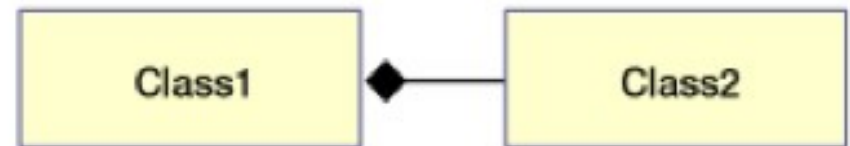
Aggregation



Multiplicity



Composition



Type of Design Patterns

- **Creational patterns** : patterns provide instantiation mechanisms, making it easier to create objects in a way that suits the situation.
- **Structural design patterns** : generally deal with relationships between entities, making it easier for these entities to work together.
- **Behavioural design patterns** : patterns are used in communications between entities and make it easier and more flexible for these entities to communicate.
- **Scope** - what the pattern applies to:
 - **Class Patterns** - Focus on the relationships between classes and their subclasses – Involve *class inheritance* reuse.
 - **Object Patterns** - Focus on the relationships between objects - Involve *object composition* reuse.

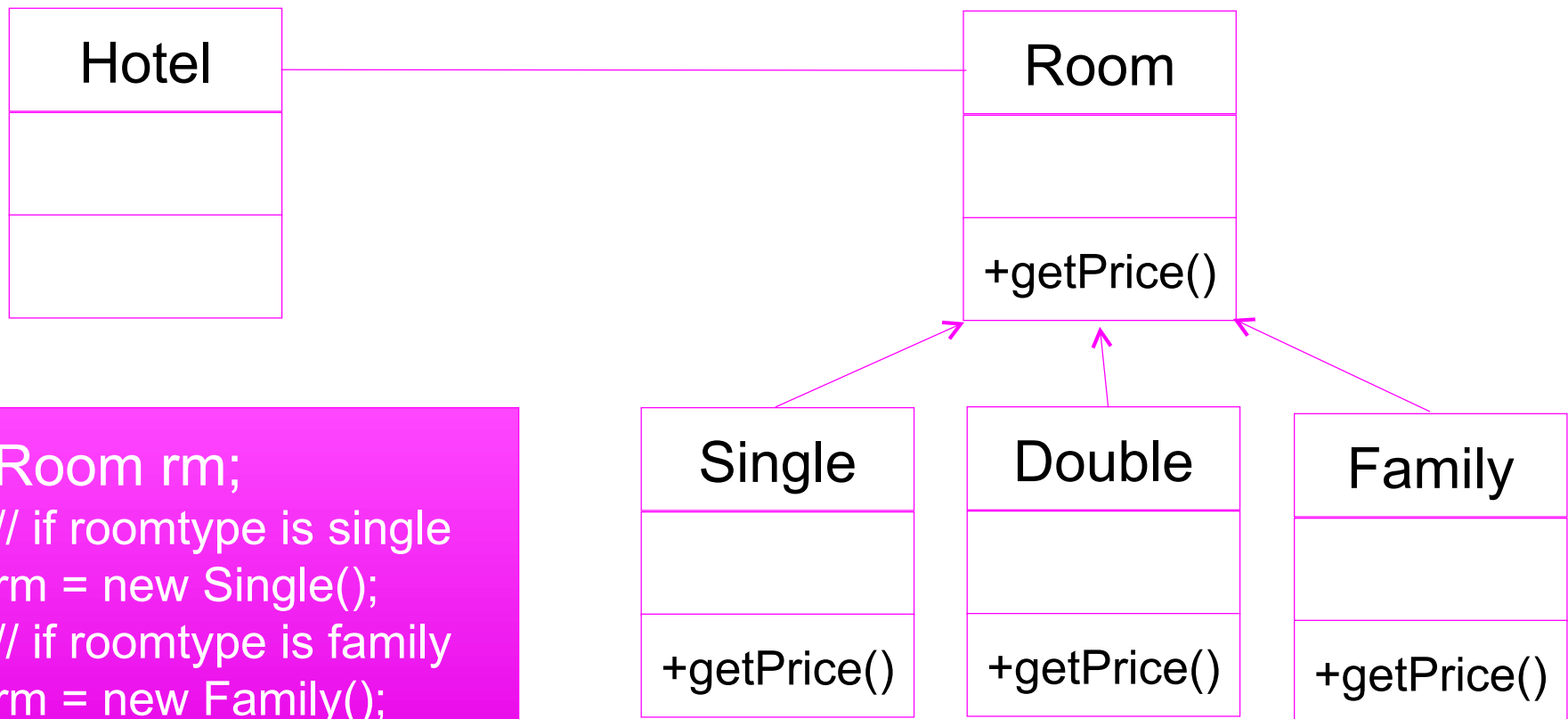
1. Creational Patterns

- This design patterns is **all about class instantiation**.
- Simplicity – some patterns make object creation easier so that clients will not contain large, complex code to instantiate an object.
- Creational patterns are ones that create objects for you, rather than having you instantiate objects directly. This gives your program more flexibility in deciding which objects need to be created for a given case.

1.1. Factory Method Patterns

- Creational Design Pattern.
- The factory related patterns are design patterns that allow for the **creation of objects without specifying the type of object that is to be created in code.**
- A factory class **contains a method** that allows determination of the created type at run-time.

Hotel Example



```

Room rm;
// if roomtype is single
rm = new Single();
// if roomtype is family
rm = new Family();
  
```

Duck Example

```
Duck duck = new MallardDuck();
```

 We want to use interfaces to keep code flexible.

 But we have to create an instance of a concrete class!

```
Duck duck;
```

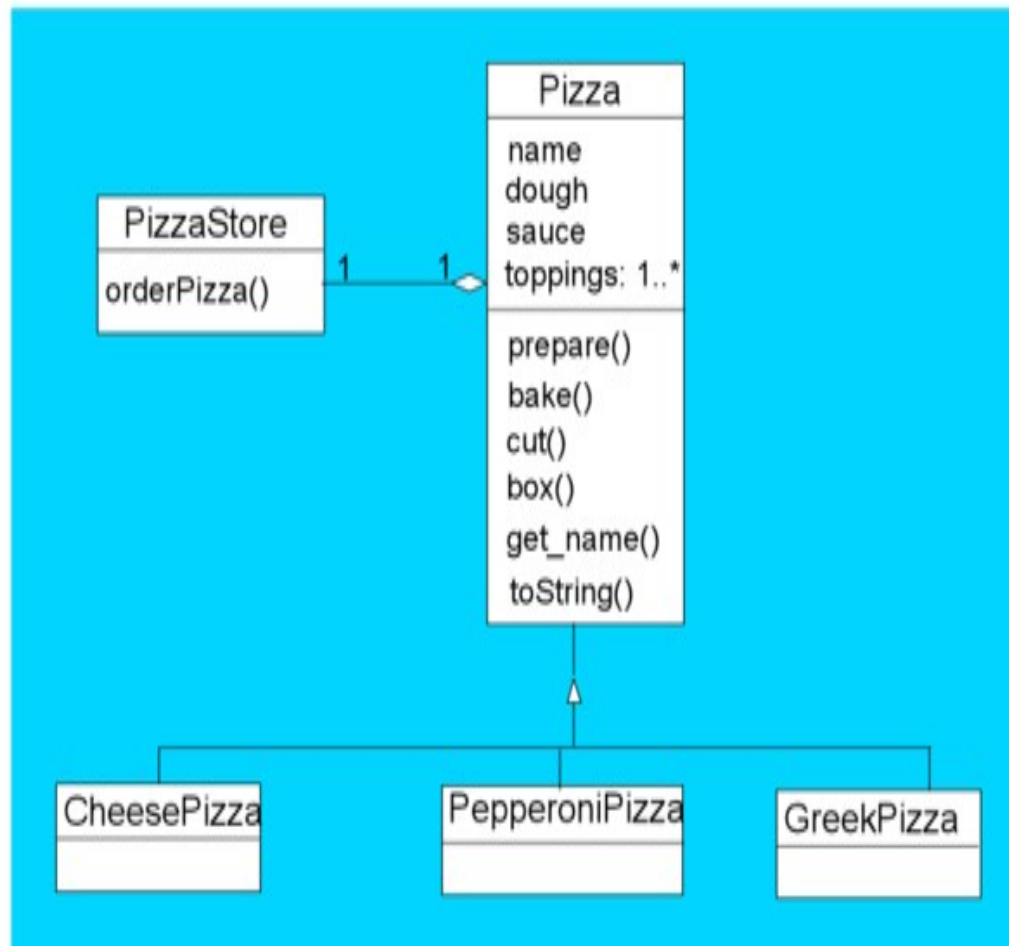
```
if (picnic) {  
    duck = new MallardDuck();  
} else if (hunting) {  
    duck = new DecoyDuck();  
} else if (inBathTub) {  
    duck = new RubberDuck();  
}
```

 We have a bunch of different duck classes, and we don't know until runtime which one we need to instantiate.

Pizza - Example

- Suppose that you are required to develop a system that accepts orders for pizza. There are three types of pizza: **cheese**, **Greek**, and **pepperoni**. The pizzas differ according to the dough used, the sauce used, and the toppings.
- Draw a class diagram for the system.

Pizza - Example



Pizza - Example

```
Pizza orderPizza(String type) {  
    Pizza pizza;
```

We're now passing in
the type of pizza to
orderPizza.

```
    if (type.equals("cheese")) {  
        pizza = new CheesePizza();  
    } else if (type.equals("greek")) {  
        pizza = new GreekPizza();  
    } else if (type.equals("pepperoni")) {  
        pizza = new PepperoniPizza();  
    }  
}
```

Based on the type of pizza, we
instantiate the correct concrete class
and assign it to the pizza instance
variable. Note that each pizza here
has to implement the Pizza interface.

```
    pizza.prepare();  
    pizza.bake();  
    pizza.cut();  
    pizza.box();  
    return pizza;
```

Once we have a Pizza, we prepare it
(you know, roll the dough, put on the
sauce and add the toppings & cheese),
then we bake it, cut it and box it!

Each Pizza subtype (CheesePizza,
VeggiePizza, etc.) knows how to
prepare itself.

```
}
```

Pizza - Example

This code is *NOT* closed for modification. If the Pizza Shop changes its pizza offerings, we have to get into this code and modify it.

```
Pizza orderPizza(String type) {
    Pizza pizza;

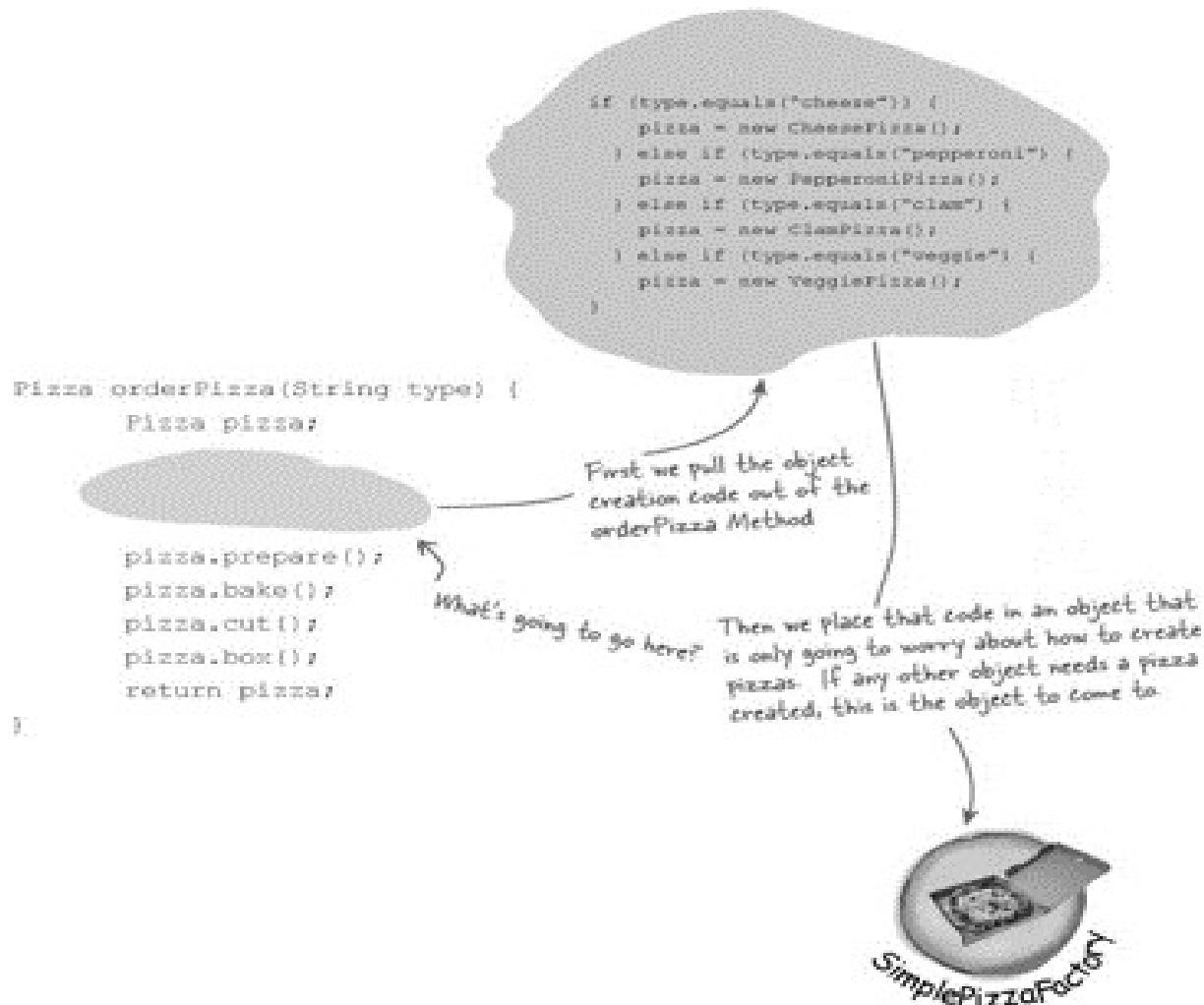
    if (type.equals("cheese")) {
        pizza = new CheesePizza();
    } else if (type.equals("greek")) {
        pizza = new GreekPizza();
    } else if (type.equals("pepperoni")) {
        pizza = new PepperoniPizza();
    } else if (type.equals("clam")) {
        pizza = new ClamPizza();
    } else if (type.equals("veggie")) {
        pizza = new VeggiePizza();
    }

    pizza.prepare();
    pizza.bake();
    pizza.cut();
    pizza.box();
    return pizza;
}
```

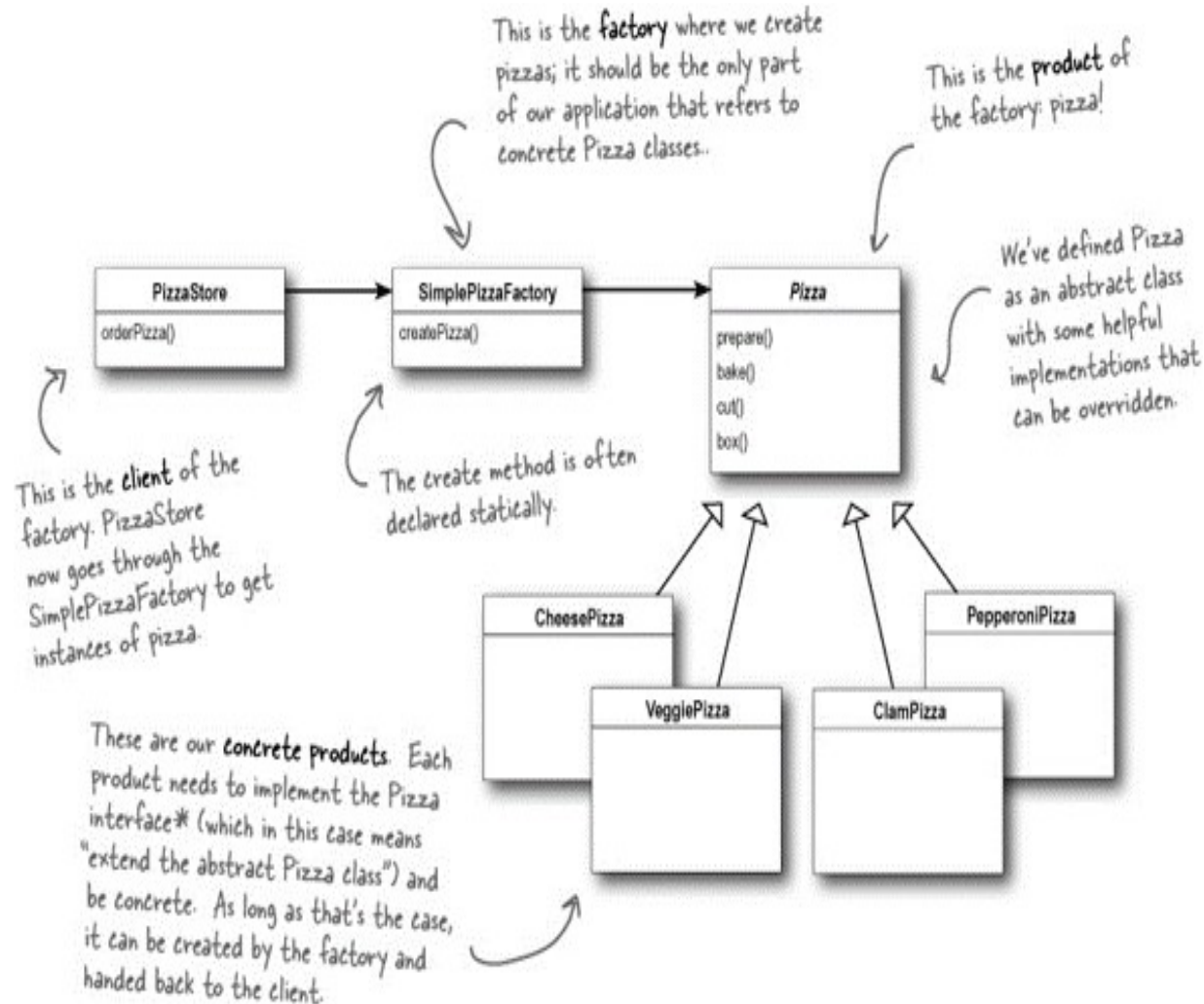
This is what varies. As the pizza selection changes over time, you'll have to modify this code over and over.

This is what we expect to stay the same. For the most part, preparing, cooking, and packaging a pizza has remained the same for years and years. So, we don't expect this code to change, just the pizzas it operates on.

Encapsulating Object Creation



Using a Simple Factory



Factory

- Factories handle the detail of object creation
- Once we have a SimplePizzaFactory, our orderPizza() method just becomes a client of that object
- Any time it needs a pizza it asks the pizza factory to make one
- The orderPizza() method just cares that it gets a pizza!

SimplePizzaFactory

Here's our new class, the SimplePizzaFactory. It has one job in life: creating pizzas for its clients.

First we define a createPizza() method in the factory. This is the method all clients will use to instantiate new objects.

```
public class SimplePizzaFactory {  
    public Pizza createPizza(String type) {  
        Pizza pizza = null;  
  
        if (type.equals("cheese")) {  
            pizza = new CheesePizza();  
        } else if (type.equals("pepperoni")) {  
            pizza = new PepperoniPizza();  
        } else if (type.equals("clam")) {  
            pizza = new ClamPizza();  
        } else if (type.equals("veggie")) {  
            pizza = new VeggiePizza();  
        }  
        return pizza;  
    }  
}
```

Here's the code we plucked out of the orderPizza() method.

This code is still parameterized by the type of the pizza, just like our original orderPizza() method was.

New SimplePizzaFactory

```

public class PizzaStore {
    SimplePizzaFactory factory;

    public PizzaStore(SimplePizzaFactory factory) {
        this.factory = factory;
    }

    public Pizza orderPizza(String type) {
        Pizza pizza;

        pizza = factory.createPizza(type);

        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();

        return pizza;
    }

    // other methods here
}

```

Now we give PizzaStore a reference to a SimplePizzaFactory.

PizzaStore gets the factory passed to it in the constructor.

And the orderPizza() method uses the factory to create its pizzas by simply passing on the type of the order.

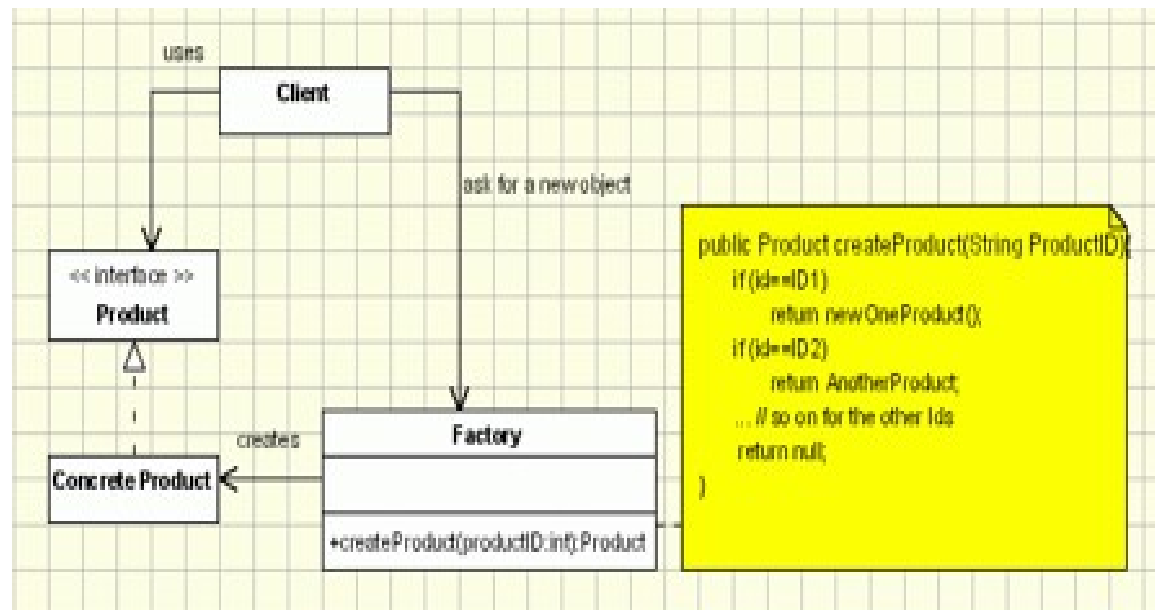
Notice that we've replaced the new operator with a create method on the factory object. No more concrete instantiations here!

Why is this better?

SimplePizzaFactory may have many clients. By encapsulating the pizza creating in one class, we now only have one place to make modifications when the implementation changes.

Simple Factory Pattern

- Intent
 - Creates objects without exposing the instantiation logic to the client
 - Refers to the newly created object through a common interface
- Implementation



2. Structural Pattern

- This design patterns is all about **Class and Object composition**.
- Structural class-creation patterns use **inheritance** to compose interfaces.
- Structural object-patterns define ways to **compose** objects to obtain new functionality.

Adapter Pattern

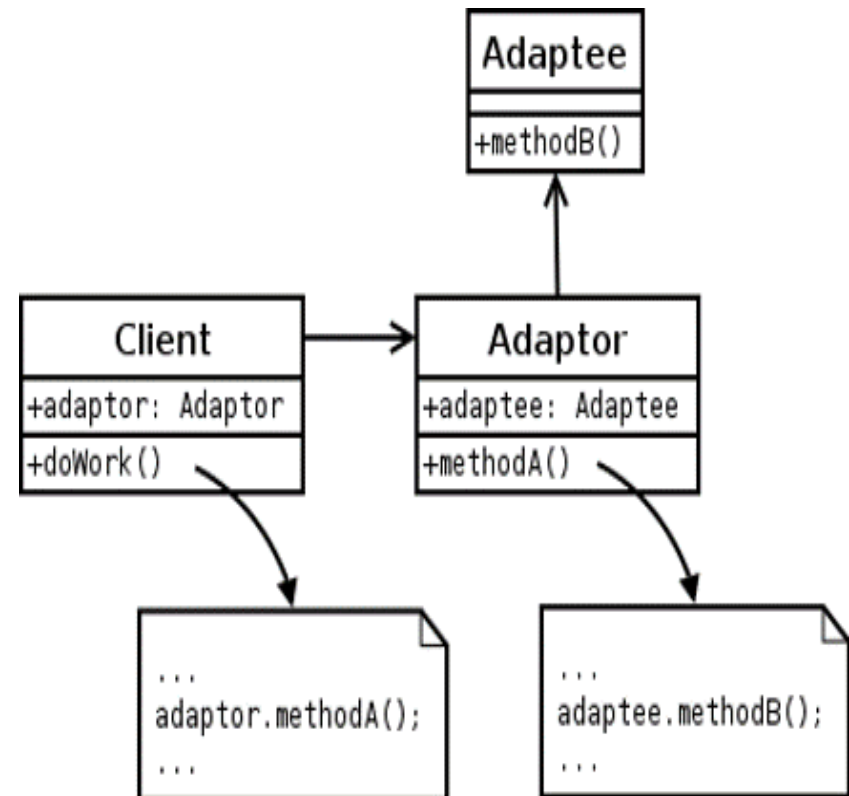
- A Structural pattern.
- Convert the **interface of a class into another interface** that the client expects. Adapter lets classes work together that could not otherwise do so, because of incompatible interfaces.
- (Essentially, we need a way to create a new interface for an object that does the right stuff but has the wrong interface).

Adapter Pattern

- **Intent:** Match an existing object beyond your control to a particular interface.
- **Problem:** A system has the right data and behavior but the wrong interface.
- **Solution:** Define an adapter class which acts as an intermediary. The adapter does little work itself but merely translates commands from one form to another.
- **Implementation:** Contain the existing class in another class. Have the containing class match the required interface and call the methods of the containing class.

Adapter Pattern – When to Use

- The Adapter pattern enables a system to use classes whose interfaces don't quite match its requirements
- Especially useful for off-the-shelf code, for toolkits, and for libraries.



Adapter Pattern - Example

```
interface Shape {
    void drawRec(int x1, int y1, int x2, int y2);
}

class LegacyRectangle {
    public void drawRec(int x, int y, int w, int h) {
        println("rectangle" + x + ', ' + y)
        println("with width " + w + " and height " + h);
    }
}
```

Adapter Pattern - Example

```
class RectangleAdapter implements Shape {  
  
    private LegacyRectangle adaptee =  
        new LegacyRectangle();  
  
    public void drawRec(int x1, int y1, int x2, int y2)  
    {  
        adaptee.drawRec(  
            x1, y1, Math.abs(x2-x1), Math.abs(y2-y1));  
        }  
    }  
}
```

Adapter Pattern - Exercise

- Write a client that would use the RectangleAdapter to draw a rectangle.

Adapter Pattern - Example

```
public class ShapeBuilding {  
    public static void main(String [] args) {  
        RectangleAdapter rectangle=  
            new RectangleAdapter();  
        int topx=10, topy=10;  
        int bottomx=110, bottomy=30;  
        rectangle.drawRect(topx, topy,  
                            bottomx, bottomy);  
    }  
}
```

Composition vs Inheritance

- Object-Orientated Programming (OOP) has **two well known candidates for the reuse of functionality**: **inheritance** (white box) and **composition** (black box).
- If you try to reuse code by inheriting from a class you will make the **subclass dependent on the parent class**. This makes the system **unnecessarily complex, less testable** and makes the exchange of functionality at run time unnecessarily hard.
- Composition means that one class uses another. You will further promote decoupling by defining the interfaces clearly. That will also give the advantage that implementations can be easily replaced.

3. Behavioral Design Pattern

- This design patterns is all about Class's objects communication.
- Behavioral patterns are those patterns that are most specifically concerned with communication between objects.
 - » Observer Design Pattern
 - » Memento Design Pattern

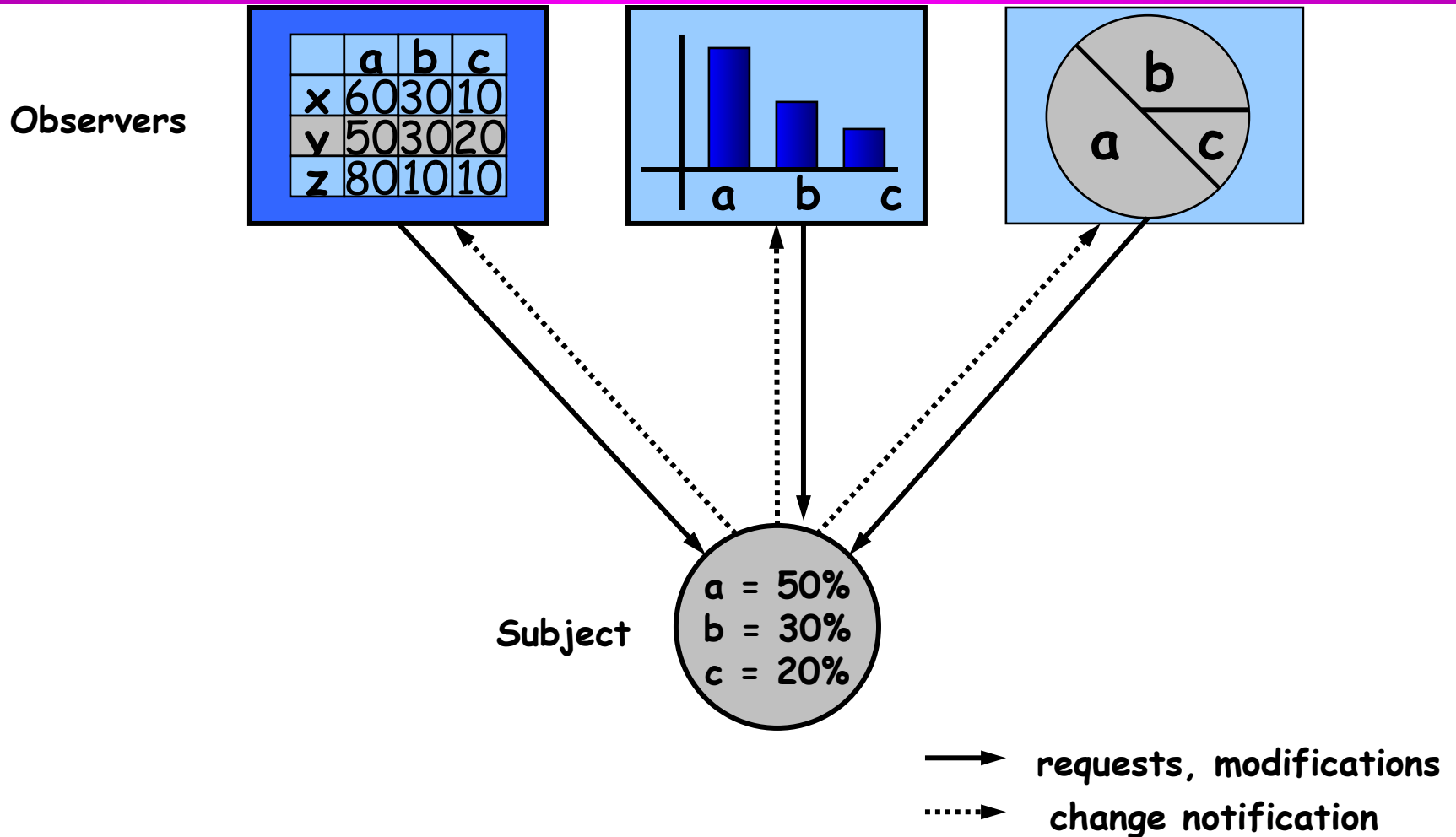
Observer Pattern

- Defines a “one-to-many” dependency between objects so that when one object changes state, all its dependents are notified and updated automatically
- a.k.a Dependence mechanism / publish-subscribe / broadcast / change-update

Subject & Observer

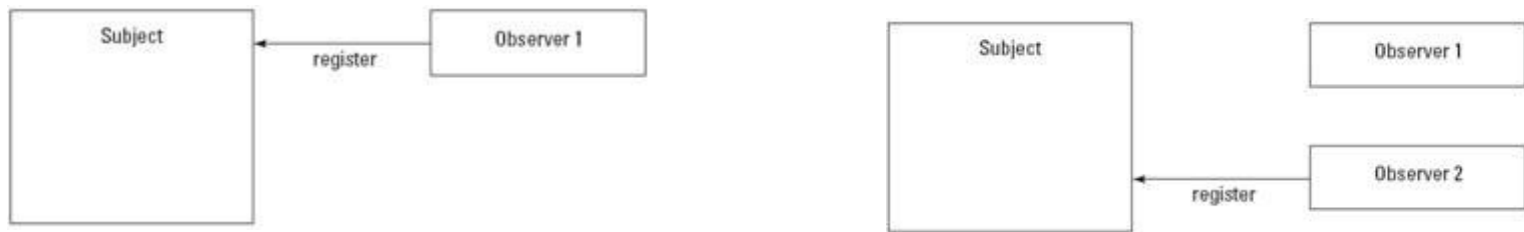
- Subject
 - » the object which will frequently change its state and upon which other objects depend
- Observer
 - » the object which depends on a subject and updates according to its subject's state.

Observer Pattern - Example

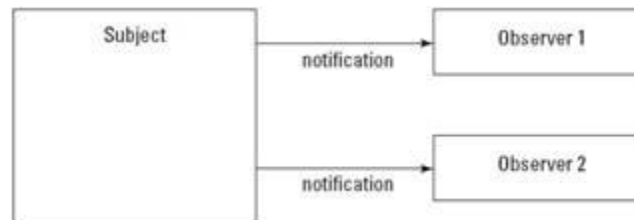


Observer Pattern - Working

A number of Observers “register” to receive notifications of changes to the Subject. Observers are not aware of the presence of each other.



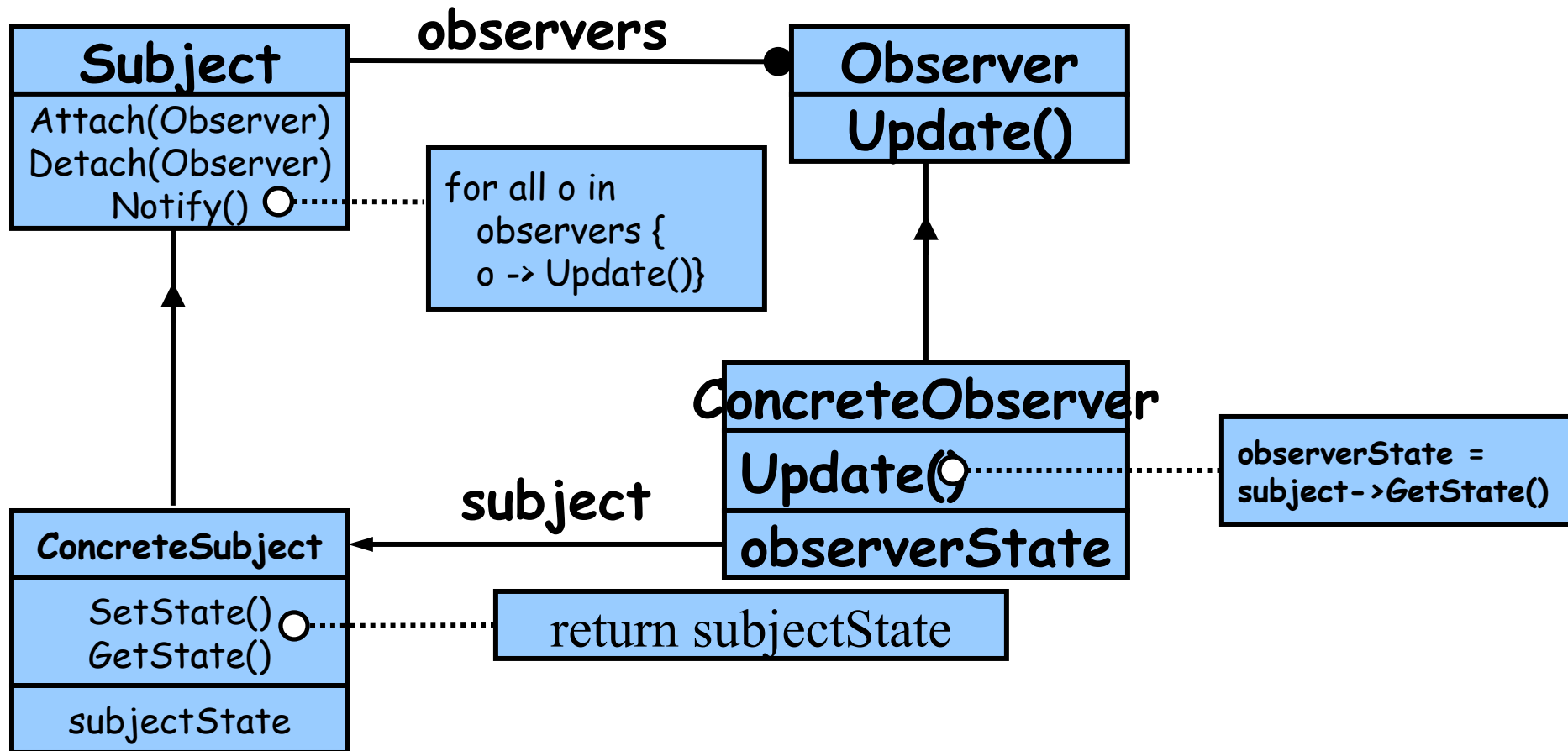
When a certain event or “change” in Subject occurs, all Observers are “notified”.



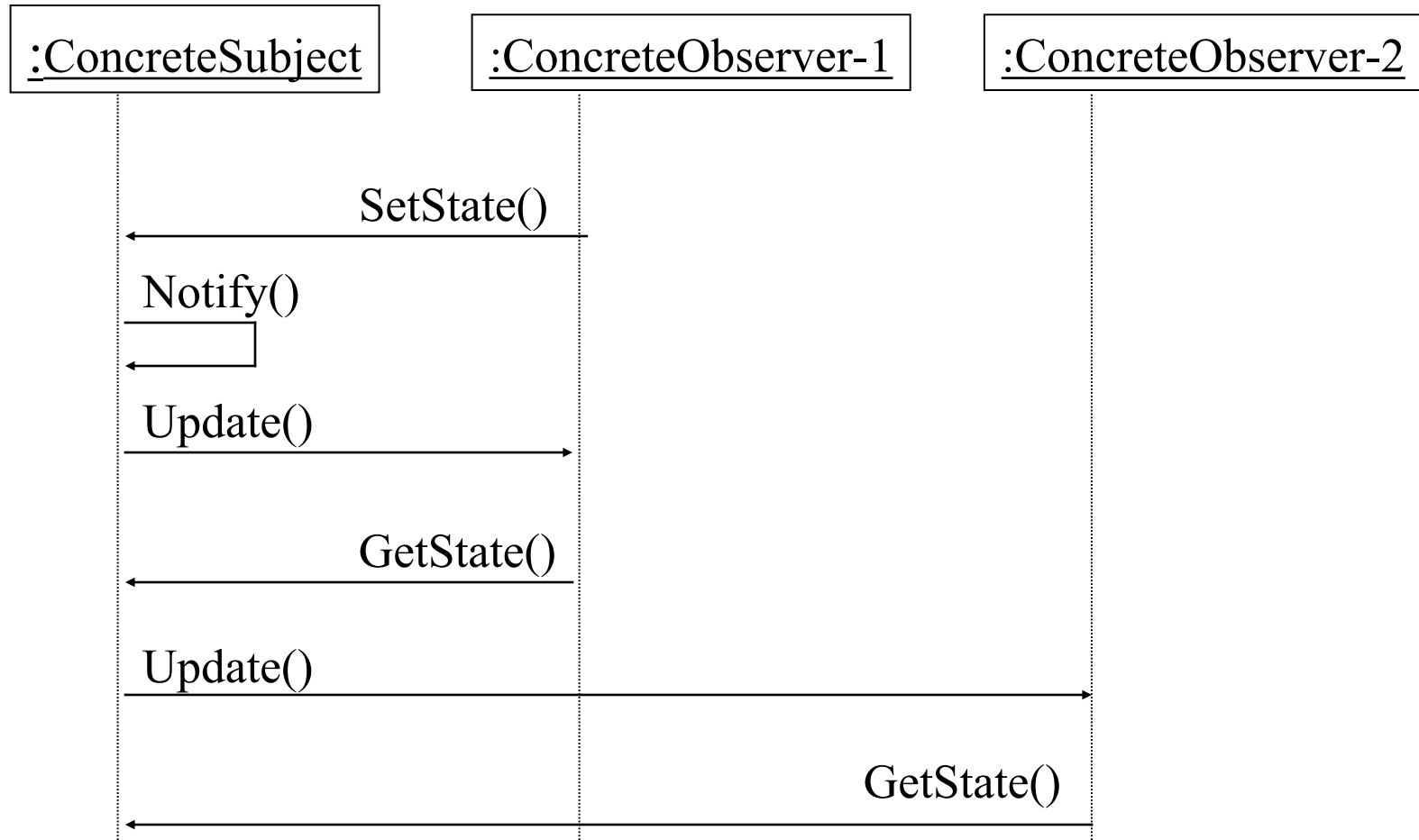
Observer Pattern - Key Players

- **Subject**
 - » has a list of observers
 - » Interfaces for attaching/detaching an observer
- **Observer**
 - » An updating interface for objects that gets notified of changes in a subject.
- **ConcreteSubject**
 - » Stores “state of interest” to observers
 - » Sends notification when state changes
- **ConcreteObserver**
 - » Implements updating interface

Observer Pattern - UML



Observer Pattern - Collaborations



Observer Pattern - Implementation

```
interface Observer {
```

```
    void update (Observable sub, Object arg)  
    // repaint the pi-chart
```

```
}
```

```
class Observable {
```

```
    public void addObserver(Observer o) {}
```

```
    public void deleteObserver (Observer o) {}
```

```
    public void notifyObservers(Object arg) {}
```

```
    public boolean hasChanged() {}
```

```
    ...
```

```
}
```



Java terminology for Subject.

Memento Pattern

- Memento pattern is **used to restore the state of an object to a previous state.**
- Memento pattern falls under the **behavioral pattern category.**

Implementation

- Memento pattern uses three actor classes.
 - » **Memento** *contains state of an object to be restored.*
 - » **Originator** *creates and stores states in Memento objects and*
 - » **Caretaker** *object is responsible to restore object state from Memento.*

Intent

- ***Capture and externalize*** an object's state without violating encapsulation.

Restore the object's state at some later time.

- » **Useful** when implementing **checkpoints** and **undo mechanisms** that let users back out of tentative operations or recover from errors.
- » Entrusts other objects with the information it needs to revert to a previous state without exposing its internal structure and representations.

Forces

- **Application needs to capture states at certain times or at user discretion. May be used for:**
 - » **Undo / redo**
 - » **Log errors or events**
 - » **Backtracking**
- **Need to preserve encapsulation**
 - » Don't share knowledge of state with other objects
- **Object owning state may not know when to take state snapshot.**

Participants

- Memento
 - » **Stores internal state of the Originator object.** Originator decides how much.
 - » Protects against access by objects other than the originator.
 - » Mementos have two interfaces:
 - Caretaker sees a narrow interface.
 - Originator sees a wide interface.

Participants (continued)

- Originator
 - » **Creates a memento containing a snapshot of its current internal state.**
 - » Uses the memento to restore its internal state.

Participants (continued)

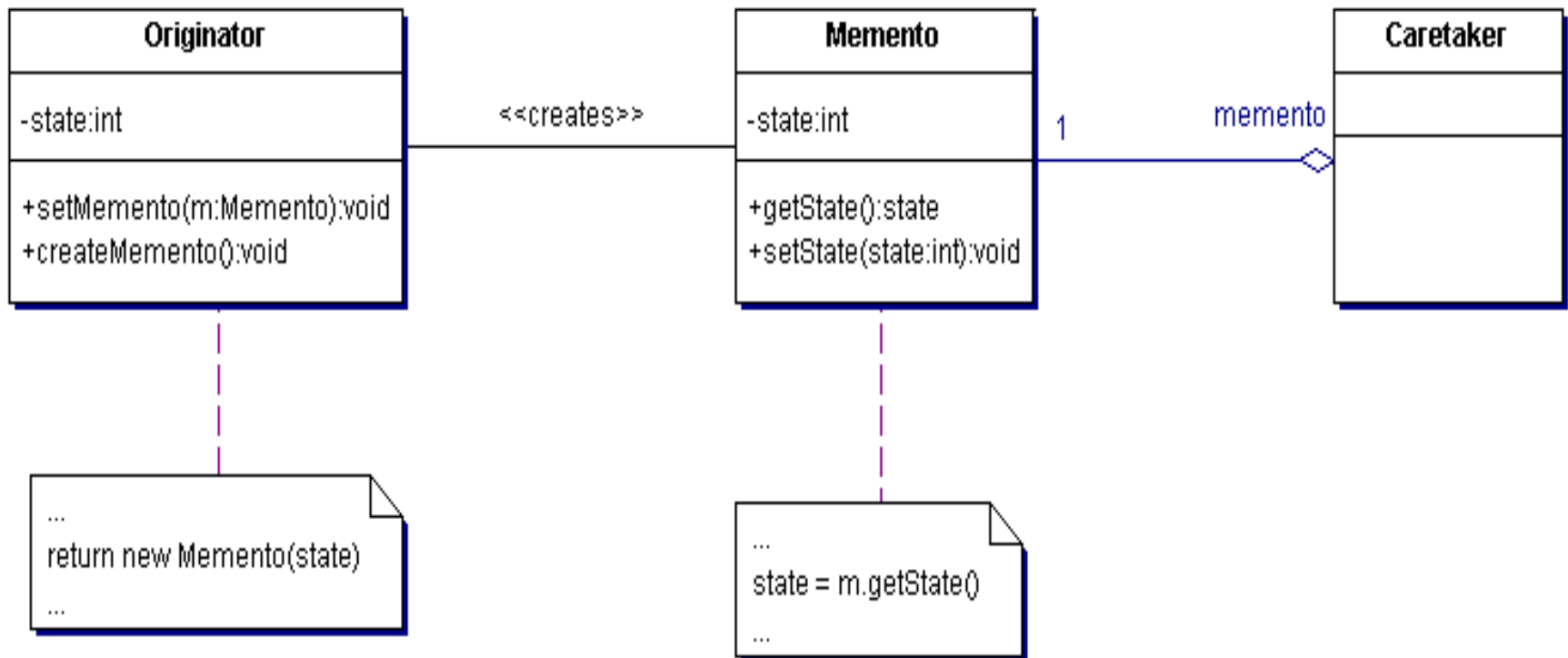
- Caretaker
 - » Is responsible for the **memento's safekeeping**.
 - » Never operates on or examines the contents of a memento.

Implementation

- When mementos get created and passed back to their originator in a predictable sequence, then Memento can save just incremental changes to originator's state.

Memento Pattern - Structure

- Intent
 - Without violating encapsulation, capture and externalize an object's internal state so that the object can be returned to this state later.
- Structure



Memento Pattern – Example

```
public class Originator {  
    private int number; private File file = null;  
    public Originator() {}  
    public Memento getMemento() { return new Memento(this);}  
    public void setMemento(Memento m) {number = m.number; file = m.file;}  
}
```

```
private class Memento implements java.io.Serializable {  
    private int number;  
    private File file = null;  
    public Memento( Originator o) { number = o.number; file = o.file;}  
}
```

Benefits of Design Patterns

- Capture expertise and make it accessible to non-experts in a standard form
- Facilitate communication among developers by providing a common language
- Make it easier to reuse successful designs and avoid alternatives that diminish reusability
- Facilitate design modifications
- Improve design documentation
- Improve design understandability